Hack the Bootsector
and Write Your Own!

This article is a tutorial on writing your own bootsector. It is a good exercise in understanding how the bootsector works and trying your hand at writing something that boots. Who knows? You might be able to write your own OS some day!

Many young programmers dream of writing the code for their own operating system. But when they realise they have to write everything from scratch, including the code for the device and file system drivers, they give up the dream.

Of course, one can just write the core components and port all the existing stuff to the new system. But even getting a kernel ready – one that supports only very basic features —would be a long process. Moreover, porting the existing components will not give you that 'developed-by-me' feeling.

So here is a solution: start off by writing something that just boots, rather than an entire OS, or even a kernel. Later, the same steps can be followed to develop an advanced bootloader, and eventually a kernel, which essentially lays the foundation for your own OS.

Before starting, let's first address the following question: why develop a new operating system?

This article doesn't assume that *everybody* is going to develop a brand new operating system. We have many OSs already, and our time and resources could be donated to such existing community projects.

However, one reason why you could consider writing your own OS is that it's a great learning experience. You are at the lowest level of software (just above the firmware), and in contact with bare metal. Any hardware component is ready to obey your orders. And all that matters is your capability to give the correct orders (which is what is called *drivers*). Despite being time-consuming, bootsector experiments can give you the confidence that doing an entire Java or PHP course can't.

Note: The code snippets and explanations given in this article target the Intel x86 architecture. This means you can try them on Intel 8086 to Core i7, any x86 compatible CPU from AMD or other vendor, or on an emulator like QEMU.

Also, modern technologies including GPT, EFI and GNU Multiboot have been avoided in favour of the traditional BIOS-based booting technology. It seems to be the best starting point, and is still supported by all modern machines.

The boot process and the bootsector

The steps involved in the boot process primarily depend on the hardware components, firmware, BIOS, disks and the operating system itself.